

Lab 6 Extra: Advanced DOM Exercise Instruction

Title: Advanced Student Result Dashboard

Level: Advanced JavaScript / WorldSkills Preparation

Task Overview

You are required to develop an **interactive Student Result Dashboard** using HTML, CSS and JavaScript. This system allows users to add multiple student records, calculate results, display the records dynamically, and manage the data using DOM manipulation and localStorage.

What You Need to Build

1. Student Input Form

Create a form that collects:

- Student Name
- Matric Number
- Assignment Mark
- Lab Test Mark
- Project Mark
- Final Exam Mark

All fields are required. Marks must be between **0 and 100**.

Use:

```
required  
min="0"  
max="100"
```

2. Add Student Record

When the user submits the form:

1. Prevent page refresh using:

```
event.preventDefault();
```

2. Retrieve all form values using `getElementById()`.
3. Convert mark values using `parseFloat()`.
4. Calculate the student's average mark.
5. Determine grade.
6. Determine status: PASS or FAIL.
7. Store the student information as an object.
8. Add the object into the students array.
9. Save the updated data into `localStorage`.

10. Display the updated student table.

3. Student Object Structure

Each student record must follow this structure:

```
{
  name: name,
  matric: matric,
  assignment: assignment,
  labTest: labTest,
  project: project,
  finalExam: finalExam,
  average: average,
  grade: grade,
  status: status
}
```

4. Calculation Requirements

Average:

```
average = (assignment + labTest + project + finalExam) / 4;
```

Grade:

```
if (average >= 80) grade = "A";
else if (average >= 70) grade = "B";
else if (average >= 60) grade = "C";
else if (average >= 50) grade = "D";
else grade = "F";
```

Status:

```
average >= 50 ? "PASS" : "FAIL";
```

Dashboard Features

5. Dynamic Student Table

Display all student records in a table.

The table must include:

No Name Matric No Average Grade Status Action

The table rows must be generated dynamically using JavaScript.

6. View Student Details

Add a **View** button for each student.

When clicked, display:

- Name
 - Matric Number
 - Assignment mark
 - Lab Test mark
 - Project mark
 - Final Exam mark
 - Average
 - Grade
 - Status
-

7. Delete Student Record

Add a **Delete** button for each student.

When clicked:

1. Remove the selected student from the array.
 2. Save the updated array to `localStorage`.
 3. Re-render the table.
 4. Update class statistics.
-

8. Search Function

Create a search box that allows users to search by:

- Student name
- Matric number

Use the `input` event so the table updates while the user is typing.

9. Filter by Status

Create a dropdown filter with:

- All
- Pass
- Fail

When the user changes the dropdown, display only matching records.

10. Sort by Highest Average

Create a button that sorts students by average mark from highest to lowest.

After sorting:

1. Update the array order.
 2. Save it to `localStorage`.
 3. Re-render the table.
-

11. Highlight Top Student

The student with the highest average should be highlighted in the table using a different background color.

12. Class Statistics

Display:

- Total students
- Number of students who passed
- Number of students who failed
- Class average
- Highest average
- Lowest average

The statistics must update whenever a student is added, deleted, or when data is cleared.

13. localStorage

Use `localStorage` so that records remain after the page is refreshed.

You must include:

```
localStorage.setItem()  
localStorage.getItem()  
JSON.stringify()  
JSON.parse()
```

14. Clear Saved Data

Create a **Clear Saved Data** button.

When clicked:

1. Ask for confirmation using `confirm()`.
 2. If confirmed, clear the array and remove data from `localStorage`.
 3. Reset the table, statistics and details panel.
-

Technical Requirements

Your solution must use:

- `addEventListener()`
 - Array of objects
 - DOM manipulation
 - `createElement()` or `innerHTML`
 - `filter()`
 - `find()`
 - `sort()`
 - `reduce()`
 - `localStorage`
 - Modular functions
-

Function Checklist

Your program should include these functions:

```
calculateAverage()  
determineGrade()  
determineStatus()  
renderTable()  
viewDetails()  
deleteStudent()  
updateStatistics()  
saveToLocalStorage()  
loadFromLocalStorage()  
applyFilterAndSearch()
```

Expected User Flow

4. User enters student information.
 5. User clicks Add Student.
 6. Student appears in the table.
 7. Class statistics update automatically.
 8. User can search, filter, sort, view details or delete records.
 9. Data remains after page refresh.
 10. User can clear all saved records.
-

Bonus Challenge

Add at least one of the following:

- Edit student record
- Dark mode
- Export table to CSV
- Duplicate matric number checking
- Toast notification after add/delete
- Responsive design for mobile screen

Submission Requirement

Submit the completed file as:

advanced_student_result_dashboard.html

Include any image/assets if used.